1. Write a C program that creates n child processes. Each child process will print its PID and its parent PID. The parent process will print its PID and the PID of each of the child processes.

Scrieti un program C care creeaza n procese fiu. Fiecare proces fiu va afisa propriul PID si PID-ul procesului parinte, iar procesul parinte va afisa propriul PID si PID-urile tuturor proceselor sale fiu.

solution

2. Write a C program that creates a linear hierarchy of n processes (a parent process creates a child process, which in turn creates a child process, and so on).

Scrieti un program C care creeaza o ierarhie liniara de n procese (procesul parinte creeaza un proces fiu, care la randul lui creeaza un proces fiu, samd.)

iterative solution
recursive solution

3. Write a C program that creates a child process. Both the parent and the child processes will run until they receive a SIGUSR1 signal. Implement signal handling such that if the parent receives the SIGUSR1 signal first, it sends it to the child process as well. If the child process receives a SIGUSR1 signal without the parent receiving the same signal, it will terminate and then the parent should correctly call wait for the child process.

Scrieti un program C care creeaza un proces fiu. Ambele procese vor rula pana cand primesc un semnal SIGUSR1. Implementati rutine pentru tratarea semnalului SIGUSR1 astfel incat, daca procesul parinte primeste semnalul SIGUSR1 il va trimite si procesului fiu. Daca procesul fiu primeste un semnal SIGUSR1 fara ca procesul parinte sa primeasca semnalul, isi va incheia executia iar procesul parinte va apela wait pentru procesul fiu.

solution

4. Write a C program that runs a bash command (potentially with arguments) received as a command line argument and times its execution.

Scrieti un program C care ruleaza o comanda bash (potential cu mai multe argumente) primita ca argument la linia de comanda si cronometreaza executia ei.

solution

5. Write a C program that implements the boltz game. Exactly N processes (numbered 1 to N, where N is given) take turns incrementing a number, starting from 1, and sending it to the next process. Process 1 starts the game by incrementing the number and sends it to process 2, which increments and sends it to process 3 and so on. Process N will send the number back to process 1, ant the cycle starts again. Each process must print the number it sends, unless the number contains the digit 7 or is divisible by 7, in which case it must print "boltz". Implement so that each process has a 1 in 3 chance to fail printing "boltz" when it should, in which case the game stops.

Scrieti un program C care implementeaza un joc de boltz. Se creeaza exact N procese (numerotate de la 1 la N, unde N este dat) incrementeaza pe rand un numar, pornind de la 1, si il trimit unui proces "vecin". Procesul 1 porneste jocul incrementand numarul si trimitandu-l procesului 2, care il incrementeaza si il trimite procesului 3, si asa mai departe. Procesul N va trimite numarul procesului 1, si ciclul se repeta. Fiecare proces va afisa numarul inainte sa il trilita, cu exceptia cazului in care numarul contine cifra 7 sau este multiplu de 7, caz in care procesul va afisa cuvantul "boltz". Ca procesele sa isi incheie executia, implementati ca un proces sa nu afiseze "boltz" intr-o situatie in care ar trebui sa afiseze "boltz", cu o probabilitate de 1/3. In acest caz, toate procesele se vor opri.

```
  Process hierarchy
  P
   | - C1
         | - C2
               | - C3
                     ...
                        | - C(n-1)

 Communication order
 P -> C1 -> C2 -> C3 -> ... -> C(n-1) -> P -> C1 -> C2 -> C3 -> ... -> C(n-1) -> ... etc.
```

solution
solution without recursion

6. Create a C program that generates N random integers (N given at the command line). It then creates a child, sends the numbers via pipe. The child calculates the average and sends the result back.

Scrieti un program C care genereaza N numere intregi aleatoate (N dat la linia de comanda). Apoi creeaza un proces fiu si ii trimite numerele prin pipe. Procesul fiu calculeaza media numerelor si trimite rezultatul inapoi parintelui.

sol

7. Write a C program that creates two child processes. The two child processes will alternate sending random integers between 1 and 10 (inclusively) to one another until one of them sends the number 10. Print messages as the numbers are sent.

Scrieti un program C care creeaza doua procese fiu. Cele doua procese isi vor trimite alternativ numere intregi intre 1 si 10 (inclusiv) pana cand unul din ele trimite numarul 10. Afisati mesaje pe masura ce numerele sunt trimise.

sol

7a. Write two C programs that communicate via fifo. The two processes will alternate sending random integers between 1 and 10 (inclusively) to one another until one of them sends the number 10. Print messages as the numbers are sent. One of the two processes must be responsible for creating and deleting the fifos.

Scrieti 2 programe C care comunica prin fifo. Cele doua procese isi vor trimite alternant unul celuilalt numere intre 1 si 10 (inclusiv) pana cand unul din ele trimite numarul 10. Afisati mesaje pe masura ce se trimit numere. Unul dintre procese va fi responsabil pentru crearea si stergerea canalelor fifo.

Process A
Process B
header.h

8. Write 2 C programs, A and B. A receives however many command line arguments and sends them to process B. Process B converts all lowercase letters from the received arguments to uppercase arguments and sends the results back to A. A reads the results, concatenates them and prints.

Scrieti 2 programe C, A si B. A primeste un numar oarecare de argumente la linia de commanda si le trimite procesului B. B converteste toate literele mici in litere mari din argumentele primite si trimite rezultatul inapoi procesului A. A citeste rezultatele procesului B, le concateneaza si le afiseaza.

Process A
Process B
common.h

9. Write two C programs that communicate via fifo. Program A is responsible for creating/deleting the fifo. Program A reads commands from the standard input, executes them and sends the output to program B. Program B keeps reading from the fifo and displays whatever it receives at the standard output. This continues until program A receives the "stop" command.

Scrieti 2 programe C care comunica prin fifo. Programul A este responsabil pentru crearea/stergerea fifo. Programul A citeste comenzi de la tastatura, le executa si trimite rezultatul programului B. Programul B citeste din fifo si afiseaza tot ce primeste de la programul A. Programele continua pana cand programul A citeste "stop" de la tastatura.

Process A
Process B
common.h

10. Create two processes A and B. A generates a random number n between 50 and 200. If it is even, it sends it to B, if it is odd it sends n+1 to B. B receives the number and divides it by 2 and sends it back to A. The process repeats until n is smaller than 5. The processes will print the value of n at each step.

Creati doua procese A si B. A genereaza un numar aleator intre 50 si 200. Daca este par, il trimite procesului B, daca nu, il incrementeaza si apoi il trimite procesului B. B imparte numarul la 2 si il trimite inapoi procesului A. Procesele repeta pana cand m este mai mic decat 5. Procesele vor afisa valoarea lui n la fiecare step.

sol

11. Create two processes A and B. A creates a shared memory segment. A then keeps reading strings from the standard input and places whatever it reads in the shared memory segment (replacing previous data). Process B, on each run, reads the data from the shared memory segment and counts the number of vowels. Process A, upon receiving a SIGINT, deletes the shared memory segment.

Creati doua procese, A si B. A creeaza o zona de memorie partajata. A citeste siruri de caractere de la tastatura si le plaseaza in segmentul de memorie partajata (inlocuind informatia precedenta). Procesul B, la o rulare, citeste informatia din segmentul de memorie si numara cate vocale sunt stocate in segment. Procesul A, la primirea unui SIGINT, sterge segmentul de memorie partajata si se incheie.

Process A
Process B
common.h

12a. Write a C program that reads a matrix of integers from a file. It then creates as many threads as there are rows in the matrix, each thread calculates the sum of the numbers on a row. The main process waits for the threads to finish, then prints the sums.

Scrieti un program C care citeste o matrice de intregi dintr-un fisier. Programul creeaza un numar de thread-uri egal cu numarul de randuri in matrice, iar fiecare thread calculeaza suma numerelor de pe un rand. Procesul principal asteapta ca thread-urile sa isi incheie executia si apoi afiseaza sumele.

12b. Same as 12a, but calculate the sum of all the elements of the matrix using as many threads as there are rows, each thread adds to the total the numbers on a row. Use the test matrix to check if the program is calculating the total sum correctly. The expected result is 1000000. Try with and without mutex.

La fel ca problema 12a dar se calculeaza suma tuturor elementelor din matrice folosind un numar de thread-uri egal cu numarul de randuri. Folositi matricea de test ca sa verificati daca rezultatul este corect. Rezultatul asteptat este 1000000. Incercati o varianta cu mutex si una fara.

Test Matrix

13. Using PIPE channels create and implement the following scenario:
Process A reads N integer numbers from the keyboard and sends them another process named B. Process B will add a random number, between 2 and 5, to each received number from process A and will send them to another process named C. The process C will add all the received numbers and will send the result back to process A. All processes will print a debug message before sending and after receiving a number.

Folosind canale PIPE implementati urmatorul scenariu:
Procesul A citeste N intregi de la tastatura si ii trimite unui proces numit B. Procesul B va adauga un numar aleator, intre 2 si 5, la toate numerele primite de la procesul A si le va trimite unui alt proces numit C. Procesul C va adauga toate numerele primite si va trimite suma inapoi procesului A. Toate procesele vor afisa un mesaj cu numerele primite si trimise.

sol

14. Write a C program (we'll refer to it as A) that creates a child process B. Process B generates one random number N between 100 and 1000. Process A keeps generating and sending random numbers between 50 and 1050 to B until the absolute difference between the number generated by A and the number generated by B is less than 50. B prints the generated numbers and all the received numbers. A will print at the end the number of numbers generated until the stop condition was met.

Scrieti un program C (numit A) care creeaza un proces fiu B. Procesul B genereaza un numar aleator intre 100 si 1000. Procesul A genereaza cate un numar aleator intre 50 si 1050 si il trimite procesului B pana cand diferenta absoluta intre numarul generat de B si numarul trimis de A este mai mica decat 50. B afiseaza fiecare numar primit de la A. A va afisa la final numarul de numere generate pana cand conditia de oprire este indeplinita.

Example:
Process B has generated 433
B received 244; difference: 189
B received 367; difference: 66
B received 723; difference: 290
B received 465; difference: 32
Process A has generated 4 numbers

sol

15. Write a program that receives strings of characters as command line arguments. For each string the program creates a thread which calculates the number of digits, the number of leters and the number of special characters (anything other than a letter or digit). The main program waits for the threads to terminate and prints the total results (total number of digits, letters and special characters across all the received command line arguments) and terminates. Use efficient synchronization. Do not use global variables

Scrieti un program care primeste la linia de comanda siruri de caractere. Pentru fiecare sir de caractere programul creeaza un thread care calculeaza numarul de cifre, litere si caractere speciale (orice nu e litera sau cifra). Programul principal asteapta ca thread-urile sa isi incheie executia si afiseaza rezultatele totale (numarul total de cifre, litere si caractere speciale din toate argumentele primite la linia de comanda) si apoi se incheie. Folositi sincronizare eficienta. Nu folositi variabile globale.

sol

16. Write a C program that receives integers as command line argument. The program will keep a frequency vector for all digits. The program will create a thread for each argument that counts the number of occurences of each digit and adds the result to the frequency vector. Use efficient synchronization.

Scrieti un program C care primeste ca argumente la linia de comanda numere intregi. Programul va calcula un vector de frecventa pentru cifrele zecimale. Pentru fiecare argument, programul va crea un thread care numara aparitiile fiecarei cifre si adauga numarul la pozitia corespunzatoare din vectorul de frecventa. Folositi sincronizare eficienta.

17. Write a C program that reads a number N and creates 2 threads. One of the threads will generate an even number and will append it to an array that is passed as a parameter to the thread. The other thread will do the same, but using odd numbers. Implement a synchronization between the two threads so that they alternate in appending numbers to the array, until they reach the maximum length N.

Scrieti un program C care citeste de la tastatura un numar N si creeaza 2 thread-uri. Unul dintre thread-uri va genera un numar par si il va insera intr-un thread primit ca parametru. Celalalt thread va genera un numar impar si il va insera in acelasi sir de numere primit ca parametru. Implementati un mecanism de sincronizare intre cele 2 thread-uri astfel incat alterneaza in inserarea de numere in sir, pana cand sirul contine N numere.

sol

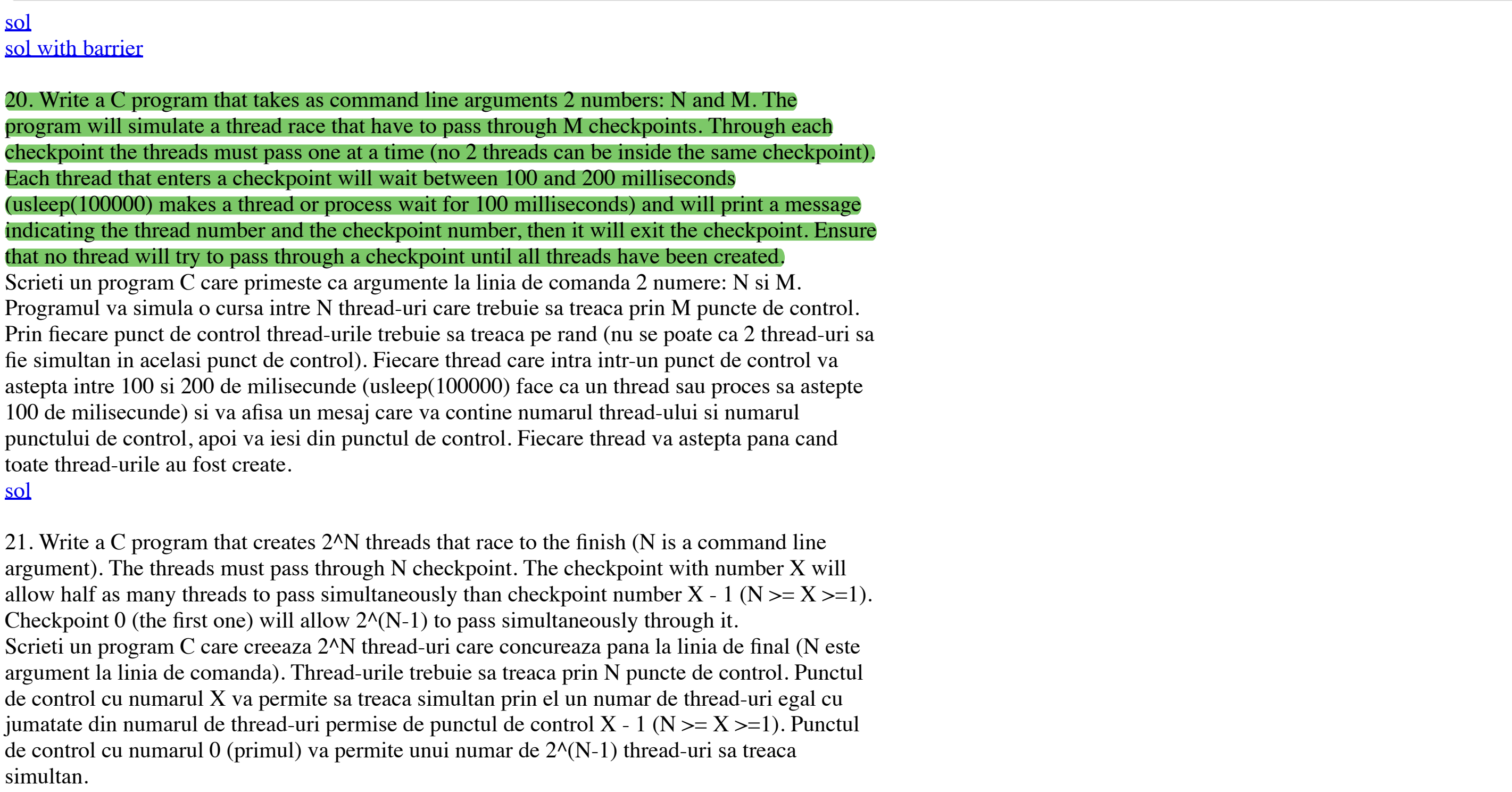
18. Create a C program that converts all lowecase letters from the command line arguments to uppercase letters and prints the result. Use a thread for each given argument.

Scrieti un program C care converteste toate literele mici din argumentele primite la linia de comanda in litere mari si afiseaza rezultatul. Folositi un thread pentru fiecare argument.

sol

19. Create a C program that takes one integer N as a command line argument, and then reads N integers from the keyboard and stores them in an array. It then calculates the sum of all the read integers using threads that obey the hierarchy presented in the image below. For any given N, the array has to be padded with extra integers with value 0 to ensure that it always contains a number of elements equal to a power of 2 (let this number be M). The required number of threads will be M - 1, let each thread have and ID from 1 to M - 1. As per the image, threads with ID >= M / 2 will calculate the sum of 2 numbers on consecutive positions in the array. Threads with an ID < M / 2 must wait for 2 threads to finish and then they will add the results produced by those two threads.

Scrieti un program C care primeste ca argument la linia de comanda un intreg N, apoi citeste de la tastatura N intregi si ii stocheaza intr-un sir. Programul calculeaza suma tuturor intregilor cititi folosind thread-uri care respecta ierarhia prezentata in imagine. Pentru orice N, sirul de intregi trebuie extins cu valori de 0 astfel incat numarul de elemente din sir sa fie o putere a lui 2 (fie numarul acesta M). Numarul de thread-uri necesare va fi M - 1, vom aloca cate un ID intre 1 si M - 1 fiecarui thread. Conform imaginii, thread-urile cu ID >= M / 2 vor calcula suma a doua numere de pe pozitii consecutive din sir. Thread-urile cu ID < M / 2 trebuie sa astepte dupa alte 2 thread-uri si apoi vor aduna rezultatul produs de cele 2 thread-uri.



sol
sol with barrier

20. Write a C program that takes as command line arguments 2 numbers: N and M. The program will simulate a thread race that have to pass through M checkpoints. Through each checkpoint the threads must pass one at a time (no 2 threads can be inside the same checkpoint). Each thread that enters a checkpoint will wait between 100 and 200 milliseconds (usleep(100000) makes a thread or process wait for 100 milliseconds) and will print a message indicating the thread number and the checkpoint number, then it will exit the checkpoint. Ensure that no thread will try to pass through a checkpoint until all threads have been created.

Scrieti un program C care primeste ca argumente la linia de comanda 2 numere: N si M. Programul va simula o cursa intre N thread-uri care trebuie sa treaca prin M puncte de control. Prin fiecare punct de control thread-urile trebuie sa treaca pe rand (nu se poate ca 2 thread-uri sa fie simultan in acelasi punct de control). Fiecare thread care intra intr-un punct de control va astepta intre 100 si 200 de milisecunde (usleep(100000) face ca un thread sau proces sa astepte 100 de milisecunde) si va afisa un mesaj care va contine numarul thread-ului si numarul punctului de control, apoi va iesi din punctul de control. Fiecare thread va astepta pana cand toate thread-urile au fost create.

sol

21. Write a C program that creates 2^N threads that race to the finish (N is a command line argument). The threads must pass through N checkpoint. The checkpoint with number X will allow half as many threads to pass simultaneously than checkpoint number X - 1 (N >= X >=1). Checkpoint 0 (the first one) will allow 2^(N-1) to pass simultaneously through it.

Scrieti un program C care creeaza 2^N thread-uri care concureaza pana la linia de final (N este argument la linia de comanda). Thread-urile trebuie sa treaca prin N puncte de control. Punctul de control cu numarul X va permite sa treaca simultan prin el un numar de thread-uri egal cu jumatate din numarul de thread-uri permise de punctul de control X - 1 (N >= X >=1). Punctul de control cu numarul 0 (primul) va permite unui numar de 2^(N-1) thread-uri sa treaca simultan.

sol

22. Write a C program that creates 10 child processes and synchronizes their execution. Each process will sleep for 1 second and then exit. Time the execution of the processes. (If all goes well, the total time should be a little over 10 seconds).

Scrieti un program C care creeaza 10 procese fiu si le sincronizeaza executia. Fiecare proces fiu va astepta 1 secunda apoi isi va incheia executia. Cronometrati durata totala a proceselor. (Daca implementarea este corecta, timpul total va fi putin mai mult de 10 secunde).

solution

23. Write a C program that receives any number of strings as command line arguments. The program creates two child processes, which inherit the parent's command line arguments (ie. no need to send the arguments via pipe/fifo to the children for this problem). Each child process creates a thread for each of the command line arguments. Each thread created by the first child will extract the vowels from its argument and will append them to a string shared among the threads. Each thread created by the second child process will extract the digits from its argument and will add them to a sum shared among the threads. Both child processes wait for their respective threads to finish and send the result to the parent via pipe. The parent displays the results.

Scrieti un program C care primeste oricate argumente la linia de comanda. Programul creeaza doua procese fiu care mostenesc argumentele de la linia de comanda primite de parinte (pentru aceasta problema nu e necesar sa se trimita argumentele prin pipe). Fiecare proces fiu creeaza cate un thread pentru fiecare argument de la linia de comanda. Fiecare thread creat de primul proces fiu extrage vocalele din argumentul sau si le concateneaza la un string accesibil de catre toate thread-urile primului proces fiu. Fiecare thread creat de al doilea proces fiu extrage cifrele din argumentul sau si le aduna la o variabila suma accesibila de catre toate thread-urile celui de-al doilea proces fiu. Ambele procese fiu asteapta ca thread-urile proprii sa se incheie, apoi trimit rezultatul catre procesul parinte prin pipe.

solution

24. Write a C program that creates N threads and one child process (N given as a command line argument). Each thread will receive a unique id from the parent. Each thread will generate two random numbers between 1 and 100 and will print them together with its own id. The threads will send their generated numbers to the child process via pipe or FIFO. The child process will calculate the average of each pair of numbers received from a thread and will print it alongside the thread id. Use efficient synchronization.

Scrieti un program C care creeaza N thread-uri si un proces fiu (N dat ca argument la linia de comanda). Fiecare thread va primi un id unic de la procesul parinte. Fiecare thread va genera 2 numere aleatoare intre 1 si 100 si le va afisa impreuna cu id-ul sau. Thread-urile vor trimite numerele generate catre procesul fiu prin pipe sau FIFO. Procesul fiu va calcula media fiecarei perechi de numere primita de la fiecare thread si va afisa rezultatul alaturi de id-ul thread-ului. Folositi mecanisme de sincronizare eficienta.

solution
solution(using FIFO because someone requested it)

25. Write a C program named A that creates 3 child processes named B, C and D. A generates a random number between 10 and 20 and sends it to process D. Processes B and C keep generating numbers between 1 and 200 and send them to process D which calculates their difference. The processes stop when the absolute difference between the numbers generated by B and C is less or equal to the number generated by process A.

Scrieti un program C numit A care creeaza 3 procese fiu numite B, C si D. A genereaza un numar aleator intre 10 si 20 si il trimite procesului D. Procesele B si C genereaza numere aleatoare intre 1 si 200 si le trimit procesului D care calculeaza diferenta lor. Procesele se opresc cand diferenta absoluta dintre numerele generate de B si C este mai mica sau egala cu numarul generat de procesul A.

solution

26. Write a C program that receives a command line argument representing a filename. The main process creates a child process. The child will read the content of the specified file, and will convert all lowercase letters preceded by the "." character to uppercase. If there are any amount of whitespaces (space, tab, newline, etc.) between the "." character and the next lowercase letter, the letter will be converted to uppercase. However, if there is any non-whitespace character between the "." character and a lowercase letter, that letter will not be changed. The child process sends to the parent the modified text. The parent process prints everything it receives from the child process.

Scrieti un program C care primeste ca argument la linia de comanda un nume de fisier. Procesul principal creeaza un proces fiu. Procesul fiu va citi continutul fisierului specificat si va converti toate literele mici precedate de caracterul "." in caractere mari. Daca exista oricate spatii albe (spatii, taburi, linii noi, etc.) intre caracterul "." si urmatoarea litera mica, acea litera trebuie convertita in litera mare. Daca exista cel putin un caracter care nu este spatiu alb intre caracterul "." si urmatoarea litera mica, acea litera nu va fi modificata. Procesul fiu trimite prin pipe parintelui textul modificat. Procesul parinte va afisa pe ecran tot ce primeste de la procesul fiu.

solution

27. Write a C program that takes two numbers, N and M, as arguments from the command line. The program creates N "generator" threads that generate random lowercase letters and append them to a string with 128 positions. The program will create an additional "printer" thread that that waits until all the positions of the string are filled, at which point it prints the string and clears it. The N "generator" threads must generate a total of M such strings and the "printer" thread prints each one as soon as it gets to length 128.

Scrieti un program C care primeste doua numere, N si M, ca argumente la linia de comanda. Programul creeaza N thread-uri "generator" care genereaza litere mici ale alfabetului aleator si le adauga unui sir de caractere cu 128 de pozitii. Programul mai creeaza un thread "printer" care asteapta ca toate pozitiile sirului de caractere sa fie ocupate, moment in care afiseaza sirul si apoi seteaza toate pozitiile sirului la NULL. Cele N thread-uri "generator" vor genera M astfel de string-uri iar thread-ul "printer" va afisa fiecare string imediat ce ajunge la lungimea 128.

solution

28. Write a C program that reads a number n from standard input and generates an array s of n random integers between 0 and 1000. After the array is created, the main process creates n + 1 threads. Each of the first n threads repeats the following steps until the array is sorted in ascending order:
- generates 2 random integers between 0 and n-1, called i and j
- if i < j and s[i] > s[j], exchanges the values of s[i] and s[j]
- if i > j and s[i] < s[j], exchanges the values of s[i] and s[j]

The n+1th thread waits until the array is sorted, after which it prints it to the console. Use appropriate synchronization mechanisms.

Scrieti un program C care citeste un numar n si creeaza un sir s de n numere aleatoare intre 0 si 1000. Dupa ce sirul este creat, procesul principal creeaza n + 1 thread-uri. Fiecare din primele n thread-uri repeta urmatorii pasi pana cand sirul s este sortat ascendent:
- genereaza 2 intregi aleatori intre 0 si n-1, numiti i si j
- daca i < j si s[i] > s[j] interschimba s[i] si s[j]
- daca i > j si s[i] < s[j] interschimba s[i] si s[j]

Thread-ul n+1 asteapta pana cand sirul este sortat, dupa care il afiseaza. Folositi mecanisme de sincronizare corespunzatoare.

solution (with barrier)
solution (with conditionals)

29. Write a C program that reads a number n from standard input and creates n threads, numbered from 0 to n - 1. Each thread places a random number between 10 and 20 on the position indicated by its id in an array of integers. After all threads have placed their number in the array, each thread repeats the following:
- Checks if the number on its own position is greater than 0.
- If yes, it substracts 1 from all numbers of the array, except the one on its own position.
- If not, the thread terminates.
- If there are no numbers in the array that are greater than 0, except the number on the thread's index position, the thread terminates.

After all threads terminate, the main process prints the array of integers. Use appropriate synchronization mechanisms.

Scrieti un program C care citeste un numar n de la tastatura si apoi creeaza n thread-uri numerotate de la 0 la n - 1. Fiecare thread pune un numar aleator intre 10 si 20 pe pozitia indicata de indexul sau intr-un sir de intregi. Dupa ce toate thread-urile au adaugat numarul in sir, fiecare thread repeta urmatorii pasi:
- Verifica daca numarul din sir de pe indexul sau este mai mare ca 0.
- Daca da, scade 1 din toate numerele din sir exceptand numarul de pe pozitia sa.
- Daca nu, isi incheie executia.
- Daca nu mai exista numere mai mari ca 0 in sir, exceptand numarul de pe pozitia sa, atunci isi incheie executia.

Dupa ce toate thread-urile isi incheie executia, procesul principal afiseaza sirul de numere. Folositi mecanisme de sincronizare corespunzatoare.

solution

30. Relay: Create a C program that reads a number n from the standard input and created 4 * n threads. The threads will be split into teams of 4. In each team the threads will be numbered from 0 and will run according to the relay rules:
- Thread 0 from each team starts, waits (usleep) for 100 and 200 milliseconds, then passes the control to thread 1
- Thread 1 waits between 100 and 200 milliseconds then passes the control to thread 2
- Thread 2 waits between 100 and 200 milliseconds then passes the control to thread 3
- Thread 3 waits between 100 and 200 milliseconds, then prints a message indicating that the team has finished, then terminates

The team from which thread 3 terminates first is considered the winning team. Use appropriate synchronization mechanisms.

Stafeta: Creati un program C care citeste un numar n de la tastatura si creeaza 4 * n thread-uri. Thread-urile vor fi impartite in "echipe" de cate 4. In fiecare "echipa" thread-urile vor fi numerotate incepand cu 0 si vor rula dupa regula stafetei:
- Thread-ul 0 din fiecare "echipa" incepe, asteapta (usleep) un timp intre 100 si 200 de milisecunde, apoi transfera controlul thread-ului 1
- Thread-ul 1 asteapta intre 100 si 200 de milisecunde apoi transfera controlul thread-ului 2
- Thread-ul 2, similar, asteapta intre 100 si 200 de milisecunde apoi transfera controlul thread-ului 3
- Thread-ul 3 asteapta intre 100 si 200 de milisecunde, dupa care afiseaza un mesaj si isi incheie executia

"Echipa" din care thread-ul 3 isi incheie executia primul este castigatoare. Folositi mecanisme de sincronizare corespunzatoare.

solution

31. Write a C program that receives a number N as a command-line argument. The program creates N threads that will generate random numbers between 0 and 111111 (inclusive) until one thread generates a number divisible by 1001. The threads will display the generated numbers, but the final number that is displayed must be the one that is divisible by 1001. No thread will start generating random numbers until all threads have been created. Do not use global variables.

Scrieti un program C care primeste un numar N ca argument la linia de comanda. Programul creeaza N thread-uri care vor genera numere aleatoare intre 0 si 111111 (inclusiv) pana cand un thread va genera un numar divizibil cu 1001. Thread-urile vor afisa numerele generate, iar ultimul numar afisat trebuie sa fie cel divizibil cu 1001. Niciun thread nu va incepe sa genereze numere pana cand toate celelalte thread-uri au fost create. Nu se vor folosi variabile globale.

solution

32. Write a C program that creates N threads (N given as a command line argument). The main process opens a file F, provided as a command line argument (the file's contents are words of a maximum of 20 characters each separated by spaces). Each thread will take turns reading between 1 and 3 words from the file and concatenating them to a thread-local buffer until all the content of the file is read. Once the whole file is completely read, the threads return their local buffer and the main process will print the result from each thread. After it does one reading pass, ensure that each thread waits for the other threads to complete their reading attempt before starting a new reading pass.

Scrieti un program C care creeaza N threaduri (N dat ca argument la linia de comanda). Programul principal deschide un fisier F, primit de la linia de comanda (fisierul contine doar cuvinte de maximum 20 de caractere, separate prin spatii). Fiecare thread va executa ture de citit intre 1 si 3 cuvinte din fisier si le va concatena la un buffer local, pana cand tot continutul fisierului a fost parcurs. Dupa ce fisierul a fost citit in intregime, threadurile retureaza bufferul lor local si procesul princial afiseaza resultatul de la fiecare thread. Dupa fiecare tura de citire, fiecare thread asteapta dupa celelalte threaduri inainte sa inceapa o noua tura de citire.

solution

33. Write a C program that creates two child process. The parent process reads one string from the keyboard (the string does not contain whitespaces) and sends it via pipe/fifo to both of its child processes. One of the child processes extracts all the vowels from the received string and the other child process extracts all the digit characters from the received string. Both child processes send what they extracted to the parent process via pipe/fifo. The parent will display the strings that are received from the child processes.

Scrieti un program C care creeaza doua procese fiu. Parintele citeste de la tastatura un string (ce nu contine spatii) si il trimite prin pipe/fifo catre ambele procese fiu. Unul din procesele fiu extrage toate vocalele din string-ul primit, iar celalalt proces fiu extrage toate cifrele din string-ul primit. Ambele procese fiu trimit string-urile extrase prin pipe/fifo catre parinte. Parintele va afisa string-urile primite de la procesele fiu.

solution pipe
solution fifo

34. Write a C program that creates two child process. The parent process reads one string from the keyboard (the string does not contain whitespaces) and sends it via pipe/fifo to both of its child processes. One of the child processes converts all the lowercase vowels from the received string to uppercase and the other child process removes all the digit characters from the received string. Both child processes send their modified string to the parent process via pipe/fifo. The parent will display the strings that are received from the child processes.

Scrieti un program C care creeaza doua procese fiu. Parintele citeste de la tastatura un string (ce nu contine spatii) si il trimite prin pipe/fifo catre ambele procese fiu. Unul din procesele fiu extrage toate vocalele din string-ul primit, iar celalalt proces fiu extrage toate cifrele din string-ul primit. Ambele procese fiu trimit string-urile extrase prin pipe/fifo catre parinte. Parintele va afisa string-urile primite de la procesele fiu.

solution pipe
solution fifo

35. Write a C program that creates two child process. The parent process reads one string from the keyboard (the string does not contain whitespaces) and sends it via pipe/fifo to both of its child processes. One of the child processes extracts the first five letters of the alphabet (a-e) from the received string and the other child process extracts the last 5 letters of the alphabet (v-z) from the received string. Both child processes send what they extracted to the parent process via pipe/fifo. The parent will display the strings that are received from the child processes.

Scrieti un program C care creeaza doua procese fiu. Parintele citeste de la tastatura un string (ce nu contine spatii) si il trimite prin pipe/fifo catre ambele procese fiu. Unul din procesele fiu extrage primele 5 litere din alfabet (a-e) din string-ul primit, iar celalalt proces fiu extrage ultimele 5 litere din alfabet (v-z) din string-ul primit. Ambele procese fiu trimit string-urile extrase prin pipe/fifo catre parinte. Parintele va afisa string-urile primite de la procesele fiu.

solution pipe
solution fifo

36. Hot Potato: Write a C program that receives a number N as a command-line argument. The main process generates a random integer between 1000 and 10000 (we'll call this variable POTATO) and creates N threads and assigns them a unique identifier starting at 1. The N threads will execute an infinite loop in which they try to subtract a random value between 10 and 100 from the POTATO and then sleep for a random amount of time between 100 and 200 milliseconds. The first thread that causes the POTATO to have a negative value prints a message that announces this alongside its given identifier, breaks the loop, and terminates. Any thread that observes that the value of the POTATO is negative will also break the loop and terminate, but without printing a message.

Hot Potato: Scrieti un program C care primeste ca argument un numar N la linia de comanda. Procesul principal genereaza un numar aleator intre 1000 si 10000 (numim aceasta variabila POTATO) si creeaza N thread-uri si le asigneaza fiecaruia un identificator unic pornind de la 1. Cele N thread-uri executa o bucla infinita in care incearca sa scada o valoare aleatoare intre 10 si 100 din POTATO si apoi asteapta un timp aleator intre 100 si 200 de milisecunde. Primul thread care face ca valoarea POTATO sa fie negativa va afisa un mesaj prin care anunta aceasta (mesajul va include si identificatorul primit de la procesul principal), apoi iese din bucla si isi incheie executia. Orice thread care detecteaza ca valoarea POTATO este negativa va iesi din bucla si isi va incheia executia, dar fara sa afiseze niciun mesaj.

potato